Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

Лабораторная работа № 2

По дисциплине “Кроссплатформенные средства разработки”

Выполнил: студент группы ИВТ-42-23

Иванов И.Л.

Проверил: кандидат технических наук

Ковалев С.В.

Чебоксары, 2026

Цель: изучить основы верстки в Flutter, создать и оформить первый экран приложения.

https://github.com/kukirmash/kukirmash_weather_app

Ход работы:

1)Установка библиотек

```
❯ flutter pub add go_router get_it talker talker_flutter
Resolving dependencies...
Downloading packages... (6.1s)
+ ansicolor 2.0.3
+ args 2.7.0
+ code_assets 1.2.1 (2.1.0 available)
+ cross_file 0.3.5+5
+ crypto 3.0.7
+ cupertino_ui 1.1.0
+ ffi 2.2.0
+ ffi_leak_tracker 0.1.2
+ file 7.0.1
+ fixnum 1.1.1
+ flutter_localizations 0.0.0 from sdk flutter
+ flutter_web_plugins 0.0.0 from sdk flutter
+ get_it 9.2.1
+ go_router 18.0.1
+ group_button 5.3.4
+ hooks 2.0.2 (2.2.0 available)
+ intl 0.20.3
+ jni 1.0.3
+ jni_flutter 1.0.3
+ jni_util 1.0.0
+ logging 1.3.0
  material_color_utilities 0.13.0 (0.13.1 available)
+ material_ui 1.3.0
  meta 1.18.3 (1.19.0 available)
+ mime 2.1.0
+ objective_c 9.5.0 (9.6.0 available)
+ package_config 3.0.0
+ path_provider 2.1.6
+ path_provider_android 2.3.1
+ path_provider_foundation 2.6.0
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
+ platform 3.2.0
+ plugin_platform_interface 2.1.8
+ pub_semver 2.2.1
+ record_use 0.6.0 (1.1.1 available)
+ share_plus 13.3.0
+ share_plus_platform_interface 7.2.0
+ talker 5.1.20
+ talker_flutter 5.1.20
+ talker_logger 5.1.20
  test_api 0.7.12 (0.7.14 available)
+ typed_data 1.4.0
+ url_launcher_linux 3.2.3
+ url_launcher_platform_interface 2.3.2
+ url_launcher_web 2.4.3
+ url_launcher_windows 3.1.6
+ uuid 4.6.0
  vector_math 2.4.0 (2.4.3 available)
+ web 1.1.1
+ win32 6.4.0
+ xdg_directories 1.1.0
+ yaml 3.1.4
Changed 50 dependencies!
8 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
○ ~/labs/7sem/CROSS/kukirmash_weather_app | main !115 ?1
```

2)Создание структуры директорий

–extensions хранит вспомогательные функции;

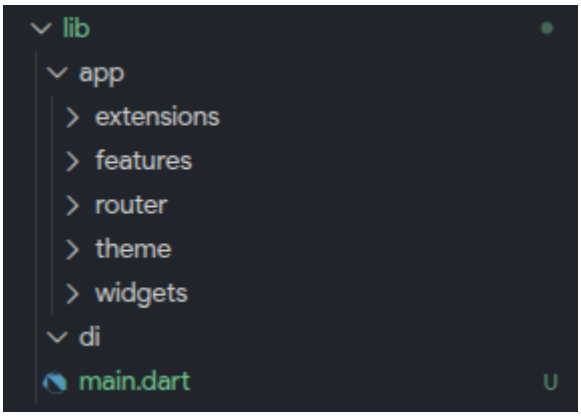
–features содержит файлы экранов и их логики;

–router отвечает за навигацию внутри приложения;

–theme – папка с настройками темы;

–widgets содержит виджеты приложения.

```
mkdir -p lib/app/{extensions,features/home,router,theme,widgets} lib/di assets/images
~/labs/7sem/CROSS/kukirmash_weather_app | main !115 ?1
```



3)Настройка инъекции зависимостей (DI) - lib/di/di.dart

Пакет `get_it` используется для реализации этой инъекции зависимостей. Библиотеки `talker` и `talker_flutter` используются в качестве логгера для отслеживания работы приложения.

```dart
import 'package:get_it/get_it.dart';
import 'package:talker_flutter/talker_flutter.dart';

final getIt = GetIt.instance;
final talker = TalkerFlutter.init();

Future<void> setupLocator() async {
 getIt.registerSingleton(talker);
}
```

4)Настройка темы приложения

Папка `theme` — это папка с настройками темы. Файл `theme_colors.dart` определяет общую цветовую палитру приложения.

Файл `theme_data.dart` обеспечивает основную конфигурацию темы, собирая цвета и стили вместе.

lib/app/theme/theme_colors.dart

```
import 'package:flutter/material.dart';

abstract final class ThemeColors {
 static const Color accent = Color.fromRGBO(20, 52, 78, 1);
 static const Color white = Color.fromRGBO(255, 255, 255, 1);
 static const Color black = Color.fromRGBO(17, 17, 17, 1);
 static const Color transparent = Color.fromRGBO(0, 0, 0, 0);
}
```

lib/app/theme/theme_data.dart

```
import 'package:flutter/material.dart';
import 'theme_colors.dart';

class AppTheme {
 static ThemeData get lightTheme {
  return ThemeData(
   useMaterial3: true,
   scaffoldBackgroundColor: ThemeColors.white,
   colorScheme: _colorScheme,
   appBarTheme: _appBarTheme,
  );
 }
}

final _colorScheme = ColorScheme.fromSeed(
 seedColor: ThemeColors.accent,
);

final _appBarTheme = const AppBarTheme(
```

```
toolbarHeight: 50,
elevation: 0,
scrolledUnderElevation: 0,
centerTitle: true,
titleTextStyle: TextStyle(
  fontSize: 20,
  fontWeight: FontWeight.bold,
  color: ThemeColors.white,
),
backgroundColor: ThemeColors.black,
);
```

файл экспорта lib/app/theme/theme.dart

```
export 'theme_colors.dart';
export 'theme_data.dart';
```

5)Папка `extensions` хранит вспомогательные функции. В нашем случае это расширение `EmptyPadding`, которое позволяет легко задавать отступы (например, `16.ph` вместо громоздкого `SizedBox`).

```
import 'package:flutter/material.dart';

extension EmptyPadding on num {
 SizedBox get ph => SizedBox(height: toDouble());
 SizedBox get pw => SizedBox(width: toDouble());
}
```

6)Папка `router` отвечает за навигацию внутри приложения. Библиотека `go_router` обрабатывает маршруты, а конфигурация навигации находится в файле `router.dart`.

```
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:talker_flutter/talker_flutter.dart';

import '../../di/di.dart';
import '../features/features.dart';
```

```
final _rootNavigationKey = GlobalKey<NavigatorState>(debugLabel:
'root');

final router = GoRouter(
 observers: [TalkerRouteObserver(talker)],
 debugLogDiagnostics: true,
 initialLocation: '/home',
 navigatorKey: _rootNavigationKey,
 routes: [
   GoRoute(
     path: '/home',
     pageBuilder: (_, state) =>
         MaterialPage(key: state.pageKey, child: const HomeScreen()),
   ),
 ],
);
```

7)**`weather_card.dart`**: В данном файле реализован переиспользуемый визуальный компонент (виджет) `WeatherCard`. Использование объектно-ориентированного подхода позволяет не передавать каждый параметр (влажность, давление, температуру) по отдельности, а передавать один объект `weather`. Виджет динамически извлекает данные из объекта и формирует текстовый блок карточки.

```
import 'package:flutter/material.dart';

/// Модель данных DailyWeather.
/// Добавлено поле imagePath для привязки конкретного изображения к
состоянию погоды.
class DailyWeather {
 final String date;
 final String condition;
 final String imagePath;
 final int tempMorning;
 final int tempDay;
 final int tempEvening;
 final int tempFeelsLike;
 final int humidity;
 final int pressure;
```

```
 DailyWeather({
   required this.date,
   required this.condition,
   required this.imagePath,
   required this.tempMorning,
   required this.tempDay,
   required this.tempEvening,
   required this.tempFeelsLike,
   required this.humidity,
   required this.pressure,
 });
}

/// Пользовательский виджет WeatherCard.
class WeatherCard extends StatelessWidget {
 final DailyWeather weather;

 const WeatherCard({super.key, required this.weather});

 @override
 Widget build(BuildContext context) {
   const imageSize = 120.0;

   return InkWell(
     borderRadius: BorderRadius.circular(16),
     child: SizedBox(
       height: imageSize,
       child: Row(
         spacing: 16,
         crossAxisAlignment: CrossAxisAlignment.start,
         children: [
           // Блок отображения иконки погоды
           ClipRRect(
             borderRadius: BorderRadius.circular(16),
             child: Image.asset(
               weather.imagePath,
               height: imageSize,
               width: imageSize,
               fit: BoxFit.cover,
               errorBuilder: (context, error, stackTrace) => Container(
                 height: imageSize,
                 width: imageSize,
                 color: Colors.grey[300],
```

```
                child: const Icon(Icons.cloud, size: 50),
              ),
            ),
          ),
          // Блок вывода текстовой информации
          Expanded(
            child: Column(
              spacing: 4,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  weather.date,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                Expanded(
                  child: Text(
                    '${weather.condition}\n'
                    'Утром: ${weather.tempMorning}°C | Днем:
${weather.tempDay}°C | Вечером: ${weather.tempEvening}°C\n'
                    'Ощущается как: ${weather.tempFeelsLike}°C\n'
                    'Влажность: ${weather.humidity}% | Давление:
${weather.pressure} мм',
                    maxLines: 4,
                    overflow: TextOverflow.ellipsis,
                    style: Theme.of(context).textTheme.bodyMedium,
                  ),
                ),
              ],
            ),
          ),
        ),
      );
  }
}
```

8)**`home_screen.dart`**: Файл главного экрана. В состояние (`State`) виджета находится массив из 7 объектов `DailyWeather`, имитирующий

получение прогноза погоды на неделю. В методе `build` свойство `itemCount` виджета `ListView.separated` привязано к длине этого массива (`weekForecast.length`), что гарантирует вывод ровно 7 карточек. Через метод `itemBuilder` каждый элемент массива последовательно пробрасывается в конструктор виджета `WeatherCard`.

```
import 'package:flutter/material.dart';

/// Модель данных DailyWeather.
/// Добавлено поле imagePath для привязки конкретного изображения к
состоянию погоды.
class DailyWeather {
 final String date;
 final String condition;
 final String imagePath;
 final int tempMorning;
 final int tempDay;
 final int tempEvening;
 final int tempFeelsLike;
 final int humidity;
 final int pressure;

 DailyWeather({
   required this.date,
   required this.condition,
   required this.imagePath,
   required this.tempMorning,
   required this.tempDay,
   required this.tempEvening,
   required this.tempFeelsLike,
   required this.humidity,
   required this.pressure,
 });
}

/// Пользовательский виджет WeatherCard.
class WeatherCard extends StatelessWidget {
 final DailyWeather weather;

 const WeatherCard({super.key, required this.weather});
```

```
@override
Widget build(BuildContext context) {
  const imageSize = 120.0;

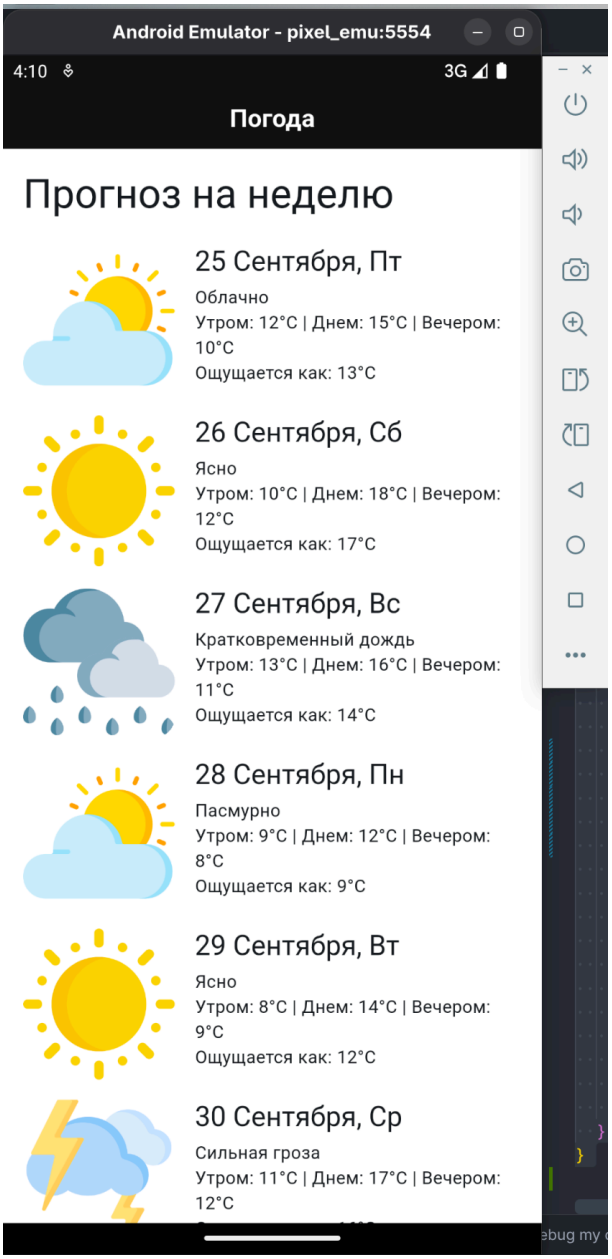
  return InkWell(
    borderRadius: BorderRadius.circular(16),
    child: SizedBox(
      height: imageSize,
      child: Row(
        spacing: 16,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // Блок отображения иконки погоды
          ClipRRect(
            borderRadius: BorderRadius.circular(16),
            child: Image.asset(
              weather.imagePath,
              height: imageSize,
              width: imageSize,
              fit: BoxFit.cover,
              errorBuilder: (context, error, stackTrace) => Container(
                height: imageSize,
                width: imageSize,
                color: Colors.grey[300],
                child: const Icon(Icons.cloud, size: 50),
              ),
            ),
          ),
          // Блок вывода текстовой информации
          Expanded(
            child: Column(
              spacing: 4,
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(
                  weather.date,
                  maxLines: 1,
                  overflow: TextOverflow.ellipsis,
                  style: Theme.of(context).textTheme.titleLarge,
                ),
                Expanded(
                  child: Text(
                    '${weather.condition}\n'
```

```
                    'Утром: ${weather.tempMorning}°C | Днем:
${weather.tempDay}°C | Вечером: ${weather.tempEvening}°C\n'
                    'Ощущается как: ${weather.tempFeelsLike}°C\n'
                    'Влажность: ${weather.humidity}% | Давление:
${weather.pressure} мм',
                    maxLines: 4,
                    overflow: TextOverflow.ellipsis,
                    style: Theme.of(context).textTheme.bodyMedium,
                  ),
                ),
              ],
            ),
          ),
        ],
      ),
    ),
  );
 }
}
```

9)Результат
Linux



Эмулятор android



Вывод: в ходе данной работы были изучены основы верстки в Flutter, создан и оформлен первый экран приложения.